

1. Introduction

In 2006, Graydon Hoare started writing a programming language which he described as “technology from the past come to save the future from itself”. Hoare’s inspirations were diverse such as CLU [44], an early object oriented language; Erlang [1], a language focusing on concurrency; and St. Andrew’s own Napier [56] language, which features polymorphism and persistence. What he came up with was the Rust language [48], first released to the public in 2012 as version 0.1 and a stable version 1.0 released in 2015.

Rust has become synonymous with its most novel feature: borrowing. This idea is not completely novel in itself: it combines *ownership types*, first described in 1998 [23], with *subtyping*, which dates back to the 70s [25]. However, the application of these ideas to memory safety was novel. In a similar vein, this dissertation aims to bring contemporary research of functional programming and dependent types into the domain of memory safety.

The primary goal of this thesis is to improve the ways in which Idris developers call or *bind to* foreign libraries (ie. libraries written in C). An Idris module, `System.ScopedIO`, was developed to support this goal by:

1. Embedding Rust-like lifetime rules within Idris’ dependent type system
2. Leveraging Idris’ `auto` implicit search to avoid “glue” code

Such that memory safety can be enforced by the compiler without having to reach for external tools or meta-languages. The design and implementation of `System.ScopedIO` is given in Section 2.

The secondary goal of this work is to build idiomatic Idris bindings to the `wgpu` graphics library. This would enable Idris developers to write graphical or massively parallel applications without having to dig into the details of FFI.

A tertiary goal is to demonstrate how tackling practical, low-level details can support theoretical research by developing an embedded domain-specific language on top of the Idris `wgpu` bindings. This goal is discussed in Section 3.2 and Section 4.3.

The rest of this dissertation is organized as follows. Section 2 describe the design and implementation of Idris modules supporting the goals above. Section 3 contains an evaluation of the implementations. And Section 4 covers additional related developments in memory safety, language approaches to removing boilerplate, and graphics programming.

1.1. Memory safety

Pointers are memory addresses with some additional type information describing the data at the pointed-to memory. Low-level languages, such as C, allow the programmer to treat pointers as integers; with arithmetic operations and conversion between different pointer [2]. Misuse of these pointer operations, and dynamic memory allocation, can lead to *memory corruption*.

For this paper, memory “safety” is defined as the absence of the the following three forms of memory corruption¹:

- Uninitialized memory read - `malloc`-ed memory and local variables do not have a defined default value. They may contain sensitive data from previous allocations; or if the value is a pointer, they likely point to invalid regions of memory.

¹These classifications are taken from [46] and [3].

- Out of bounds read/write - reading and writing to an address past what was asked for in `malloc`. This may lead to exposure of sensitive data, unrelated values being manipulated, or program crashes. It should be noted that the out-of-bounds address could be a valid allocation, for example if `malloc` returned adjacent chunks of memory in separate calls, but this is still considered unsafe.
- Use after free, double free², and memory leaks³ - `free(3)` [5] marks an allocation as no longer used, so the chunk of memory can be reallocated to a user program or used for bookkeeping. Thus, using memory after a `free` can read or write unknown data or corrupt the allocator itself.

Each of these issues can be solved by language design decisions. However, solving correct use of `free` generally incurs a performance penalty. Respectively, the accepted solutions for the above forms of memory corruption are:

- Uninitialized reads can be solved by the language defining default values (as in Go) or by enforcing all variables to be initialized with a value (as in Java).
- Out of bounds reads can also be solved relatively easily by bounds checking array access and disallowing pointer arithmetic. In C, they are both equivalently used to access arbitrary memory relative to a pointer. The runtime cost of bounds is eased by CPU branch prediction or avoided entirely by compiler optimizations [26]. The methods for avoiding out-of-bounds memory access is described in Section 2.1.2 and implemented in Section 2.2.4.
- `free`-related bugs are not as easy to solve. Historically, they're only avoided with costly *garbage collection*⁴ (eg. Go, Java, C#) or avoiding dynamic allocation entirely. However, Rust has broken this Pareto frontier, being memory safe without incurring the cost of garbage collection through its concept of ownership and borrowing. A full description is out of scope of this work, but in short Rust values *owned* by certain parts of the program, and may be *borrowed* as *references* to the owned value. Importantly, references have *lifetimes* which cannot be longer than the owned object they reference. Lifetimes are checked at compile time such that references never refer to memory that has been `free`-ed or gone out of scope. Lifetimes are further described in Section 2.1.1 and an implementation is described in Section 2.2.3.

High-level languages such as Idris are usually memory safe, employing the methods above. However, FFI punches a hole through these safety mechanisms; there's no guarantee the foreign code will maintain the invariants and bookkeeping of the host language. Having a safety gap isolated to FFI is better than an entire language, but the severity and frequency of memory corruption bugs [17, 46, 51] motivates further exploration of memory safety in FFI. In Section 2, I present such an exploration, resulting in a library for building memory safe FFI bindings.

²These generally fall under the class of *dangling pointers*, however this super-class also includes pointers to program stack memory; for the purposes of this paper this is not considered since the targeted Idris backend, `chez` scheme, only gives the programmer access to heap allocation (ie. `malloc/free`)

³Memory leaks are not always considered unsafe [4], it's mentioned here as prevention is useful and is solved along with the other forms.

⁴Garbage collection involves automatically tracking and scanning objects in the heap for liveness, this adds significant overhead both in memory use and CPU usage.

2. System.FFI.Safe

2.1. Design

2.1.1. Preventing memory leaks and use-after-free with scopes

The design of lifetimes in this library is heavily based on the `ST` (state thread) monad [53]. Informally, `ST` allows mutable state (generally disallowed in pure functional programming) by keeping track of the thread which the mutable state belongs to. Note, the “thread” here should not be confused with concurrency primitive.

```
data ST : (s : Type) -> (a : Type) -> Type where
  MkSt : (s -> (a, s)) -> ST s a

data STRef : (s : Type) -> (a : Type) -> Type where
  MkRef : (s : Type) -> a -> STRef s a

newSTRef    : a -> ST s (STRef s a)
readSTRef   : STRef s a -> ST s a
writeSTRef  : STRef s a -> a -> ST s ()

runST : ({ s : Type } -> ST s a) -> a
```

Listing 1: `ST`, `STRef`, and associated types as defined in [53]

The important detail is that `runST` requires `{s : Type} -> ST s a`. The `ST` value being run needs to be general over all types `s`. Calling `runST` on an `ST` then “materializes” the general `s` into a specific type, which we’ll denote `s1`; `s1` marks the thread. Thus, a call to `newSTRef` inside the `ST s1 _` becomes `ST s1 (STRef s1 a)`. The above implementation hides some category-theoretical magic (ie. Monads), but assume the `STRef s1 a` inside `ST` is freely accessible; latter calls to `readSTRef` and `writeSTRef` take up the materialized `s1` from their inputs into their output `STs`. These specific `ST s1 _` types cannot be used in a different `runST` because the `s1` is no longer general enough for `runST`.

```
failing "When unifying"
testRunSTBad : Int
testRunSTBad = runST $ readSTRef (runST $ newSTRef 2)
```

This pattern has a couple useful properties. Firstly, the computation inside an `ST` starts and ends within `runST` function. If allocation were to occur inside the `ST`, this is a convenient place for deallocating such memory. Secondly, the specific `STRef s1 a`, is only usable within the thread `s1`. If `STRef` held a pointer to memory allocated inside `ST`, it would then not be accessible after being deallocated at the end of `runST`. These two properties prevent the free related errors as defined in Section 1.1.

Together, another interpretation of `ST` is that it behaves like a scope: `STRef s` are only usable within the scope. This usability is enforced by the type checker.

2.1.1.1. Lifetimes

Sadly, `ST` cannot be used as-is. One important pattern we’d like to support is to freely allow a longer lived allocation to be used where shorter a shorter lived allocation is expected. As

demonstrated, references in `ST`s are mutually exclusive. We'll have to modify `ST` such that they can have associated *subscopes*, and a way to use this relation to shorten the lifetime of reference from the parent scope into the lifetime of the subscope.

In order to support the sharing of references between subsopes Rust treats lifetimes types with a *subtyping* relationship [6]. Using subtyping means these relationships are worked without automatically, and a reference with longer lifetime can be used, without any extra syntax, where shorter lifetimes are expected. Idris does not explicitly feature subtyping. However, the combination of dependent types and `auto` implicits allows programs to be written in a similar way. Lifetimes and the type for subsoping relations are described in Section 2.2.3.

2.1.2. Preventing out of bounds access

Another design goal is preventing out-of-bounds and uninitialized memory access. This requires two separate pieces:

- Ensuring the correct amount of memory is allocated.
- Reading and writing to that memory in a controlled manner.

The `CType : Type -> Type` GADT is responsible for the former. It behaves like a sealed interface [7] restricting the set of Idris types which need to be handled by FFI-related functions. In a Curry-Howard perspective `CType a` is proposition that the type `a` can be represented in C. The `CType a` itself does not specify what this representation is. Rather, the representations are provided by functions such as `sizeof`, `alignof`, and `Marshal`. Ultimately this design means specifying valid C types and allocating them can be done in plain Idris, without supporting code for each possible C type (eg. each different `typedef`-ed `struct`).

Inspired by Haskell's `Storable` [21], the `Marshal` and `Unmarshal` types are responsible for controlling memory access. `Marshal` corresponds to memory writes, and Haskell's `poke`, while `Unmarshal` corresponds to memory reads and Haskell's `peek`.

```
-- haskell storable
class Storable a where
  sizeof      :: a -> Int
  alignment   :: a -> Int

  peekElemOff :: Ptr a -> Int -> IO a
  pokeElemOff :: Ptr a -> Int -> a -> IO ()

  peekByteOff :: Ptr a -> Int -> IO a
  pokeByteOff :: Ptr a -> Int -> a -> IO ()

  peek      :: Ptr a -> IO a
  poke      :: Ptr a -> a -> IO ()

  destruct   :: Ptr a -> IO ()
```

Listing 2: The Haskell `Storable` typeclass from [8]. Note that as a typeclass, each FFI-boundary-crossing type needs to implement it. That is, any programmer wishing to make a type cross FFI boundaries needs to write code making direct memory accesses.

2.1.3. Leveraging dependent types and `auto` implicit search

One of the main developments of this work is that this library employs `auto` implicit search for generating `Marshal` instances for compound types (ie. `struct`) on the fly by the compiler. This is an improvement over existing techniques which require external tools for generating this “glue” code (see Section 4.2).

Additionally, dependent types are used throughout to define memory safety properties which the compiler can verify.

2.2. Implementation

2.2.1. Preamble

Idris provides some types supporting FFI, namely the pointers `Ptr a` and `AnyPtr`, and `Struct`. Firstly I’ll redefine the the pointer types for clarity. The `[external]` pragma makes these *phantom types*. Since they don’t have constructors Idris may make assumptions that a `Ptr a` and an `AnyPtr` have no inhabitants (as in `Falsity`); with `[external]`, Idris treats these types as inhabitable but *opaque*. `Ptr` is the type of addresses, or *pointers*, which point to the type `a` in memory. `AnyPtr` is simply a memory address with no assumptions about the pointee; this is equivalent to `void*` in C.

```
public export
data Ptr : Type -> Type where [external]

public export
data AnyPtr : Type where [external]
```

Next, `Structs` are defined. Again, this is a phantom type, which will be represented in Idris as a pointer to the heap allocation of the `Struct`. That is, a variable of type `Struct "t" [("f", Int)]` will contain a memory address to a heap allocated `struct t`. This may seem odd since the type is `Struct` and not `Ptr (Struct ...)`, but this *boxing* [35] of values is a common practice in function programming languages; it is needed here because Idris does not expose the local variable stack.

```
public export
data Struct : String -> List (String, Type) -> Type where [external]
```

`Struct` is a dependent type indexed by a `String`, the C `struct`’s name, and a list of fields. Fields have a `String` field name and a `Type`. The `Struct`’s name and field names won’t be used here but it serves as documentation and a way of differentiating between `Structs` with similar types.

2.2.2. Allocation

The polymorphic types above have one major flaw; *any* Idris can be accepted where a `Type` is required. For example `Struct "list" [("t", Type), ("values", AnyPtr)]` is a valid Idris type, but C does not have a `Type` type because is not even polymorphic! Thus `CType a` will serve as a proposition than a type `a` can be represented in C. Its limited set of constructors then serve as proofs that a limited set of `as` can be represented in C. This limited set can be split into primitives like, `CInt` and `CDouble` and recursive constructors `CPtr`, and `CStruct`.

More concretely, `CInt` is a proof that `Int` has a C representation. The recursive definitions similarly state that, for example, given a proof that some type `t` has a C representation, `Ptr t` has a C representation.

The `CStruct` constructor uses the `All` list quantifier to represent a proof of all fields in a `Struct` having proofs of C representation (ie. `CType a` for all field types `a`). This, in turn, can be used to prove that the `Struct` has a C representation.

For brevity, the rest of this section will only consider the types `Int`, `Double`, `Ptr a` and `Struct name fields` as C types. But this can be extended to a full range of C types including sized signed and unsigned integers, arrays, and function types.

```
public export
data CType : Type -> Type where
  CInt      : CType Int
  CDouble   : CType Double
  CPtr      : CType t -> CType (Ptr t)
  CStruct   : All (CType . Builtin.snd) fields -> CType (Struct name fields)
```

With the C types defined, the functions which actually define the representation (ie. their size and alignment) can be defined. These can then be used for allocation and deallocation.

```
sizeof  : (repr : CType a) -> Int64
alignof : (repr : CType a) -> Int64

alloc   : (repr : CType a) -> IO (Ptr a)
free    : Ptr a -> IO ()
```

2.2.3. Lifetime and subscopes

Lifetimes are implemented as a wrapper around the `s : Type` of `ST`. The two constructors reflect the two ways of running our desired scoped computation: it can run at the top level `runScopedIO`, or it can run within a subscope `runSubScopedIO` where the constructor references a parent scope.

```
public export
data Lifetime : Type where
  LRoot : (θ thr : Type) -> Lifetime
  LSub  : (θ thr : Type) -> (θ _ : Lifetime) -> Lifetime
```

The hierarchy implicit in `Lifetime` is not enough to fully reason about them. Another type `AtLeastAsLong` is needed, representing the proposition that a lifetime `a'` is at least as long as `b'`. A proof of this proposition can then be used to shorten a `Ref a'` into the shorter-lived `Ref b'`. `AtLeastAsLong` type has two constructors representing base cases and one recursive case. The base cases are `ALALSame`, which serves as proof that a lifetime is at least as long as itself, and `ALALParent`, which serves as proof that a lifetime `a'` is at least as long as a direct descendant of itself. Lastly, the recursive `ALALTrans` admits a transitive property for the `AtLeastAsLong` relation.

```

public export
data AtLeastAsLong : Lifetime -> Lifetime -> Type where
  ALALSame      : AtLeastAsLong a' a'
  ALALParent    : AtLeastAsLong a' (LSub b a')
  ALALTrans     : AtLeastAsLong a' b' -> AtLeastAsLong b' c' -> AtLeastAsLong a' c'

```

2.2.4. Marshal/Unmarshal

The `Marshal` type is inspired by the `poke` function in Haskell’s `Storable` [21]. They both take a pointer to a C allocation, host value, and execute a side-effecting computation. However, there are a few differences. Primarily, in `Marshal` the Idris type and C type do not have to be the same. This is necessary for enforcing memory safety: `Ref a' a` can be marshaled into a `Ptr a` without actually realizing the `Ptr a`. That is, it’s written to memory through a `Ptr (Ptr a)`. This is an important property, because it means a programmer cannot stumble into a function which creates a pointer from a reference⁵ this conversion does have to happen but can be safely encapsulated inside a `Marshal` instance and not exposed to the programmer.

Unmarshal follows in a similar manner. However, as a design choice, unmarshaling of structures is not provided. The motivation behind this decision is that entire structures should be passed around as `Refs`. Bringing the entire value into Idris suggests, incorrectly, that they will be kept up to date according to the backing data, which is not possible nor safe.

⁵This functionality *is* provided but with an apt warning in the name: `unsafeRefPtr`

```

public export
data Marshal
  : Lifetime -> ity -> cty -> Type where
  MkMarshal :
    { auto repr : CType cty } ->
    (ity -> Ptr cty -> IO Int64) -> Marshal a' ity cty

export
%hint
marshalInt : Marshal a' Int Int
marshalInt = MkMarshal $ \x, ptr =>
  primIO $ prim_ptrWrite32 (prim_forgetPtr ptr) (cast x)

export
%hint
marshalDouble : Marshal a' Double Double
marshalDouble = MkMarshal $ \x, ptr =>
  primIO $ prim_ptrWrite64 (prim_forgetPtr ptr) (prim_DoubleToBits x)

export
%hint
marshalRef :
  { auto repr : CType t } ->
  Marshal a' (Ref a' t) (Ptr t)
marshalRef = MkMarshal $ \x, ptr =>
  primIO $ prim_ptrWrite64
    (prim_forgetPtr ptr)
    (prim_PtrToBits $ prim_forgetPtr $ unsafeRefPtr x)

```

Listing 3: `Marshal` and non-recursive instances. The actual implementations are provided for demonstration. In these functions, `x` is the value to be written, and `ptr` is a pointer to the location they should be written to. These function ultimately end up writing to a raw pointer.

```

export
%hint
marshalStructBase :
  { 0 ia : Type } ->
  { auto repr : CType ca } ->
  Marshal a' ia ca ->
  Marshal a' ia (Struct name [(f, ca)])

export
%hint
marshalStructRec :
  { auto repr : CType ca } ->
  { auto reprs : All (CType . Builtin.snd) cb } ->
  Marshal a' ia ca ->
  Marshal a' ib (Struct name cb) ->
  Marshal a' (Pair ia ib) (Struct name ((field, ca)::cb))

```

Listing 4: Recursive instances of `Marshal` for structs. Their implementations are not given here due to space constraints, but can be found in Section 6.1.1. In short, `marshalStructRec` writes one field to the destination, calculates an offset for the next field, while considering alignment, and recurses on the `Marshal` for the other fields (either a `marshalStructRec` or a `marshalStructBase`).


```

public export
data Unmarshal
  : Lifetime -> ity -> cty -> Type where
  MkUnmarshal :
    { auto repr : CType cty } ->
    (Ptr cty -> IO ity) -> Unmarshal a' ity cty

export
%hint
unmarshalInt : Unmarshal a' Int Int

export
%hint
unmarshalDouble : Unmarshal a' Double Double

```

Listing 5: `Unmarshal` and instances

2.2.5. Scoped allocations

The `ScopedIO` monad can now be defined. The constructor is convoluted, but in short: it is a wrapper around `IO`, the type representing functions with side effects, returning some `a` paired with a list of clean up functions `List (IO ())`. An early implementation considered returning a list of pointers for `free` to be called on. However, it is often the case that additional code needs to run before `free`-ing an object, this is known as a *destructor* [59] and part of a C++ design pattern called “resource acquisition is initialization” (RAII). Rust captures this concept with the `Drop` [9] trait. In `System.ScopedIO`, a primitive function `defer` allows programmers to add arbitrary actions to the scope which will run at its end. `defer` is used in a following section to build something the more closely resembles destructors, `safeFFIDrop`.

`runScopedIO` looks similar to `runST`, but with a `Lifetime` instead of arbitrary `Type`. Additionally there is a `runSubScopedIO` which passes a proof of the subscope relationship, a `AtLeastAsLong`, which can be used to `readRef` from the outer scope.

2.2.5.1. Getting struct fields

As mentioned, reading entire `Structs` is not supported. Instead a `Ref` to a struct can be *projected* to a `Ref` of one of its fields. This requires a new type `Field` representing a proposition that an `fname : String` is a valid field of the struct. Additionally, a type level function `FieldType` uses a proof of the proposition to extract the type of the field. Note that `Field` is provided to `getField` as an `auto` implicit, so when calling `getField` the proof of `Field` rarely has to be provided.

```

export
data ScopedIO : Lifetime -> Type ->
Type where
  MkScopedIO :
    IO (Pair (List (IO ())) a) ->
    ScopedIO a' a

public export
data Ref : Lifetime -> Type -> Type
where [external]

public export
runScopedIO :
  HasIO io =>
  ((0 a' : Lifetime) -> ScopedIO a' a)
->
  io a

public export
runSubScopedIO :
  (0 a' : Lifetime) ->
  (
    (0 b' : Lifetime) ->
    (0 p : AtLeastAsLong a' b') ->
    ScopedIO b' a
  ) -> ScopedIO a' a

```

```

public export
newRef :
  { auto m : Marshal a' ity cty } ->
  ity ->
  ScopedIO a' (Ref a' cty)

public export
writeRef :
  { auto m : Marshal a' ity cty } ->
  ity -> Ref a' cty ->
  ScopedIO a' Int64

public export
readRef :
  { auto unm : Unmarshal a' ity cty } -
  >
  { auto 0 p : AtLeastAsLong a' b' } ->
  Ref a' cty ->
  ScopedIO b' ity

export
defer : IO () -> ScopedIO a' ()

```

Listing 6: The full API for allocating, reading, writing, and automatically deallocating memory needed to do FFI

```

public export
data Field : String -> List (String, Type) -> Type where
  First : Field name ((name, ty)::fs)
  Later : Field name fs -> Field name (f::fs)

public export
0 FieldType : Field fname fs -> Type
FieldType (First { ty }) = ty
FieldType (Later l) = FieldType l

public export
getField :
  { auto reprs : All (CType . Builtin.snd) fields } ->
  Ref a' (Struct name fields) -> (fname : String) ->
  { auto f : Field fname fields } ->
  Ref a' (FieldType f)

```

Listing 7: The API for accessing `Struct` fields through a `Ref`. `Field` i

2.2.6. Shimming existing FFI

There is one last problem to be solved. There are some misalignment between `CType`s and the existing expectations of FFI. For example, functions returning C's `void` are represented in Idris as returning `()`. `()` should not be a valid `CType` since C does not have tuples, nor should `void` as it cannot be field in a `struct`. Another example is `Refs`, with `Refs` to `Struct`s being particularly problematic: plain `Struct`s cannot be created so a compiler backend intrinsic

`prim_derefStruct` is necessary. Recall that Idris does not give us access to the local variable stack, so `prim_derefStruct` does not really do a dereference, the underlying value is still a pointer. However, the plain `Struct` Idris type signals to the backend that the *generated code* should dereference the pointer prior to making the function call to C, when the stack can be accessed.

This shimming is done via the `FFICall` type, representing yet another proposition. `FFICall` is a proposition that a list of arguments can be used to call a function; the proofs have slightly different types, ie. `Ref _ (Struct ...)` can be provided to a function `Struct ... -> ...`. This type is used in `safeFFI` function, again as an `auto` implicit, to *shim* or adjust the function call accordingly.

```
public export
data FFICall
  : (0 a' : Lifetime) ->
    List Type ->
    (fty : Type) ->
    Type where [search fty]

FCReturn :
  { b : Type } ->
  CType b =>
  FFICall a' [] (PrimIO b)

FCReturnVoid :
  FFICall a' [] (PrimIO ())

FCSame :
  CType a =>
  FFICall a' args f ->
  FFICall a' (a::args) (a -> f)

FCRefPtr :
  CType a =>
  FFICall a' args f ->
  FFICall a' ((Ref a' a)::args) (Ptr
a -> f)

FCDerefStructRef :
  All (CType . Builtin.snd) fields =>
  FFICall a' args f ->
  FFICall a' ((Ref a' (Struct name
fields))::args) (Struct name fields ->
f)
```

```
public export
FFICallRet : FFICall a' args f -> Type
FFICallRet (FCReturn { b }) = b
FFICallRet (FCReturnVoid) = ()
FFICallRet (FCSame rest) = FFICallRet
rest
FFICallRet (FCRefPtr rest) = FFICallRet
rest
FFICallRet (FCDerefStructRef rest) =
FFICallRet rest
```

Listing 8: `FFICall` proposition type which defines the how each type is shimmed. And `FFICallRet` for extracting the return value from an `FFICall` proof. Table 1 below explains each construtor.

```

-- NOTE: chez specific!!
prim_derefStruct : Ptr (Struct name fields) -> Struct name fields
prim_derefStruct = believe_me

-- sanity check that our ref has the expected lifetime
checkRef : Ref a' x -> ScopedIO a' ()
checkRef _ = pure ()

export
safeFFI :
  { auto call : FFICall a' args f } ->
  f -> HList args -> ScopedIO a' (FFICallRet call)
safeFFI = go call
  where
  go :
    (call' : FFICall a' args' f') ->
    f' -> HList args' -> ScopedIO a' (FFICallRet call')
  go FCReturn      f _      = primIO f
  go FCReturnVoid  f _      = primIO f
  go (FCSame rest) f (a::arg) = go rest (f a) arg
  go (FCRefPtr rest) f (a::arg) = do
    _ <- checkRef a
    go rest (f $ unsafeRefPtr a) arg
  go (FCDerefStructRef rest) f (a::arg) = do
    _ <- checkRef a
    go rest (f $ prim_derefStruct $ unsafeRefPtr a) arg

```

Listing 9: `safeFFI`, a function for making FFI call safely. Note that it does not use normal function application syntax, instead it uses an `HList`. The `Return`-related definitions simply return the value as it exists. While the definitions take an `a` out of the argument `HList` and apply it to the function `f`, after shimming the argument as needed.

cons name	value given to foreign function	value expected by foreign function	notes
<code>FCReturn</code>	n/a	n/a	A proof for functions returning C types
<code>FCReturnVoid</code>	n/a	n/a	A proof for functions returning <code>void</code> in C, and <code>()</code> in Idris
<code>FCSame</code>	A C-representable type, <code>a</code>	The same type <code>a</code>	These arguments are the status quo for Idris FFI
<code>FCRefPtr</code>	A <code>Ref</code> to a C-representable type <code>a</code>	A pointer to <code>a</code>	Converts a <code>Ref</code> to a pointer and applies the function to the pointer. Note that the lifetime <code>a'</code> of the <code>Ref</code> is observed in the return type of <code>FFICall</code> . That is, this proof carries the lifetime of the <code>Ref</code> and is used by <code>safeFFI</code> to make sure all <code>Ref</code> arguments are valid for the scope.
<code>FCDerefStructRef</code>	A <code>Ref</code> to a C-representable <code>Struct</code>	A pointer to the <code>Struct</code>	Signal to the backend the the underlying <code>Struct</code> pointer will need to be dereferenced before calling the foreign function.

Table 1: An easier to understand mapping table of of the `FFICall` constructors

2.2.7. Destructors Redux

Above, `ScopedIO` ensures the memory we allocate into a scope is deallocated at the end of the scope; and, that this memory cannot be accessed outside of the scope. However, some objects will not be directly allocated in Idris. Rather, a foreign library may provide a constructor function for both allocating and initializing the object. This would also be paired with destructor, described above in Section 2.2.5. `defer` is not enough to safely handle this lifecycle because the value returned from the foreign constructor (and then destructed) can still be used at the end of the scope. However, if the value returned by the foreign constructor is a pointer, it can be used as a `Ref`, as if returned from `alloc`/`prim_malloc`.

Safe destruction in this way, is provided by `safeFFIDrop`. It is similar to `safeFFI`, but with the following differences:

- It requires a destruction function.
- The FFI function must return a pointer; enforced with the `IsPtr` type.

- `safeFFIDrop`, then, returns a `Ref` to the pointed-to value.

One unsolved problem that is that calling the destructor via `safeFFI` within the body of the scope results in a double-free at the end of scope, when the automatic destructor is run. That is to say two “safe” operations, one `safeFFIDrop` and one `safeFFI`, can result in memory corruption. This issue is discussed in Section 5.1.

```
public export
data IsPtr : Type -> Type where
  MkIsPtr : IsPtr (Ptr a)

-- A type-level function returning the pointed-to type of a pointer
public export
0 UnPtr : (a : Type) -> { auto 0 prf : IsPtr a } -> Type
UnPtr _ =
  case prf of
    MkIsPtr { a = b } => b
    _ => assert_total $ idris_crash "unreachable!!"

unsafeIsPtrRef : a -> { auto prf : IsPtr a } -> Ref a' (UnPtr a)
unsafeIsPtrRef x =
  case prf of
    MkIsPtr => unsafePtrRef x

export
safeFFIDrop:
  { auto call : FFIcall a' args f } ->
  { auto isptr : IsPtr (FFICallRet call) } ->
  f ->
  (drop : FFIcallRet call -> IO ()) ->
  HList args ->
  ScopedIO a' (Ref a' (UnPtr (FFICallRet call)))
safeFFIDrop fn drop args = do
  ret <- safeFFI fn args
  defer (drop ret)
  pure $ unsafeIsPtrRef ret
```

Listing 10: The function `safeFFIDrop` and helpers `IsPtr` and `UnPtr`. `IsPtr` is used to prove the return value of the foreign function is a pointer. `UnPtr` is used for getting the pointed-to type when describing the new returned `Ref`.

3. Evaluation

This section evaluates if, and to what extent, the work herein achieves the goals enumerated in Section 1:

1. Improving Idris’ FFI (Section 3.1)
3. Demonstrating how improvements of practical details can “trickle up” into research domains. (Section 3.2)

A snippet of the demo program, `src/Examples/TriangleSys.idr`, mentioned throughout this section is in Section 6.2.

3.1. System.ScopedIO Evaluation

`System.ScopedIO`, is evaluated with respect to:

- The functionality provided beyond Idris’ current FFI.

- How strongly memory safety is maintained

3.1.1. FFI

Firstly, `ScopedIO` cannot entirely remove the need for generating Idris code. Especially in a large library, such as `wgpu`, a script was needed to translate the definitions of a C header file into equivalent definitions in Idris. This results in a large Idris file which needs to be parsed, typechecked, and tracked by version control. Anecdotally, Idris’ incremental compilation handled this gracefully, only recompiling when the cache folder was deleted or a change to the file was made. Other possible approaches, with various tradeoffs, for automatically binding to C are described in Section 4.2.

```
testScope1 : IO Int
testScope1 = runScopedIO $ \a' => do
  x <- newRef { cty = Int } $ the Int 10
  runSubScopedIO a' $ \b', _ => do
    runSubScopedIO b' $ \c', _ => do
      readRef x

failing "Can't solve constraint"
testScope2 : IO Int
testScope2 = runScopedIO $ \a' => do
  x <- runSubScopedIO a' $ \b', _ => do
    newRef { cty = Int } $ the Int 10
  readRef x
```

Listing 11: A correct and incorrect usage of `ScopeIO` and `Refs`. In `testScope1`, the outer `Ref`, `x`, is automatically shortened via `auto` implicit search of `AtLeastAsLong a' c'`. Such a proof is found for the `readRef` and it is immediately returned to the outer scope(s). In `testScope2`, the inner scope smuggles a `Ref` out of the scope, but fails to `readRef` on it in the outer scope.

The `ScopedIO` library minimizes the amount of code required to use FFIs in Idris. With direct access to memory allocation and shim provided by `FFICall`, no C glue code is needed to allocate, read, or write structs. Instead, we leverage the compiler’s `auto` implicit mechanism is able to write some of this glue code and extend Idris’ current FFI to call foreign functions using our memory safe references.

```

TestStruct : Type
TestStruct = Struct "TestStruct"
  [ ("a", Bits8)
  , ("b", Bits16)
  , ("c", Bits32)
  , ("d", Bits64) ]

%foreign "C:printStruct,mylib"
printStruct : TestStruct -> PrimIO ()

testSafeFFI : IO ()
testSafeFFI = runScopedIO $ \a' => do
  x <- newRef { cty = TestStruct }
  ( 101
  , 10203
  , 20405060
  , 50404020205 )
  safeFFI printStruct [x]

```

Listing 12: Example usage of `safeFFI` using a `Ref _ (Struct ...)` to call a foreign function `printStruct` which takes the `Struct` by value.

One downside of over-using `auto` implicits in this way, is that it has detrimental effect on compile times. The demo program used in a following Section 3.1.2, is around 500 lines of code but takes 10 seconds to compile on an Apple MacBook Pro with an M2 10-core CPU. One reason for this may be that `auto` implicit search does not only look for a *single* construction satisfying the type, but also a sort of “trivial uniqueness” of the implementation [27]. This means, Idris continues to search the entire, bounded, search space even if a solution is found. The Idris compiler provides some timing statistics via the `--timing` flag but it did not provide enough granularity to implicate `auto` implicits precisely.

During development of the demo program I found explicit type annotations were sometimes needed in calls to `newRef` as the typechecker could not manage the ambiguity of `Marshal`. The particular case which triggered this is the allocation of a `Struct` which has a field with another `Struct` type (see Listing 13). Creating this type requires “nested” calls to `newRef`; the parent `Struct` is created using a tuple where one of the elements is also created by `newRef`. In this scenario, the typechecker needs to find a single type that is a suitable output of the inner `newRef` and a suitable input to the outer `newRef`; that is to say the compiler has no information about this type from either the function that produces it (the inner `newRef`) or the function consuming it (the outer `newRef`). Not having sufficient information, the typechecker gives up. Annotating the inner `newRef` function call with the expected C type resolves the ambiguity.


```

failing "Can't find an implementation"
  nestedNewRef : ScopedIO a'
  (Ref a'
    (Struct "hello"
      [ ("f", Int)
        , ( "g" , Struct "world" [ ("h", Int) , ("i", Int) ])
      ])
    )
  nestedNewRef = do
    newRef
      ( 25
        , !(newRef
            -- uncommenting the following line fixes the
            -- ambiguity issue
            -- { cty = Struct "world" [("h", Int), ("i", Int)] }
            ( 50
              , 75
            )
          )
      )

```

Listing 13: This function fails to type check with the error

`Can't find an implementation for Marshal a' ...`

In a similar vein, resolving type errors in calls `safeFFI` and `newRef` can be difficult. When an incorrect argument is passed, the typechecker complains that a `Marshal` instance can't be found and may dump all of the expected input and output types. This is a worse experience in comparison to plain function calls or record construction, where the typechecker can tell the user exactly which argument is incorrect.

Idris has a feature for resolving some of these issues called “determining arguments” [10], but I could not find a way to use them to fully eliminate type annotations. Ultimately, while `auto` implicits result in code that is easier to read, it comes with the cost of making it harder to write.

3.1.2. Memory safety

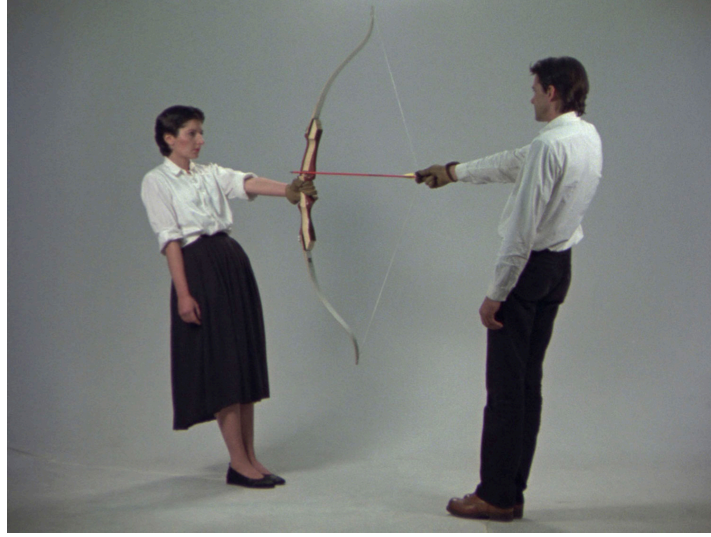


Figure 1: No matter how well `ScopedIO` may protect *us* from making mistakes around memory usage, building software requires trusting the developers of external libraries to not make the same mistakes. This idea was depicted in *Rest Energy* [47], a performance where Marina Abramović and Ulay drew a bow using their body weight, with an arrow loaded and pointed at Abramović’s heart.

In order to verify the logic of `ScopedIO`, a small demo program was run under `valgrind`’s `memcheck` tool. `valgrind` is “a framework for building dynamic analysis tools” [11]; that is, tools which inspect the behavior of a program as it runs. The `memcheck` tool in particular tracks program allocations and memory accesses to identify errors. `valgrind`’s coverage of our safety criteria (defined in Section 1.1) is as follows:

- Uninitialized memory in general is not reported, rather an error is reported when uninitialized memory “affect[s] your program’s externally-visible behaviour” [12]. For example, if a jump or move CPU instruction depends on the uninitialized value.
- Out-of-bound access is tracked in two ways. The simple case of unallocated memory is tracked using “V” bits (ie. “valid” bits) for the address space [12]. The more complicated case of misusing a pointer to access an arbitrary allocation is not handled in general. However, `memcheck` can identify some of these errors by through a *redzone*: by default the tool will allocate 16 bytes before and after an allocation, and set the “V” bits to invalid [13]. This means linear access which are off-by-one will be caught, rather than possibly continuing to an adjacent chunk of allocated memory.
- Free-related errors, again, come with a caveat. `memcheck` can track that memory allocations are `free`-ed exactly once. However, a chunk of memory may be reallocated such that a new *variable* happens to hold the same address value as an old variable. `free`-ing this allocation with the old variable is incorrect but `memcheck` cannot distinguish between variables.

In order to focus on the design of `ScopedIO`, the demo program uses `Graphics.WGPU.Sys` rather than the higher-level `Graphics.WGPU`. The demo program can be found in `src/Example/TriangleSys.idr` in the artifacts repository. An initial run of the program resulted in hundreds of errors. These were mostly memory leaks originating from transient dependencies⁶.

⁶ie. Dependencies of `wgpu`

These are out of our control and were thus suppressed via `valgrind`'s `--suppressions` flag. Further, some memory leaks originated from initialization functions for Linux's graphics drivers; these leaks persisted despite double-checking the de-initialization functions were being called.

Once external errors were suppressed, `valgrind` illuminated two issues with `ScopedIO`. An out-of-bounds access of one byte, and memory leaks from `wgpu` resources. The out of bounds read was caused by the following helper function which creates a manually managed string from an Idris `String`:

```
stringRef : String -> ScopedIO a' (Ref a' Bits8)
stringRef s = do
  let len = strLength s
  ptr <- primIO $ prim__malloc . (+ 1) $ cast len
  defer (primIO $ prim__free ptr)
  primIO $ prim__memcpyStr ptr s (cast len)
  pure $ unsafePtrRef $ prim__castPtr ptr
```

The mistake present in this function is negligence of the NUL-terminator. In C, strings are expected to end in a NUL ASCII character represented by byte with value 0. Below is a diff of `stringRef` which fixes the issue.

```
- ptr <- primIO $ prim__malloc $ cast len
+ ptr <- primIO $ prim__malloc . (+ 1) $ cast len
  defer (primIO $ prim__free ptr)
  primIO $ prim__memcpyStr ptr s (cast len)
+ _ <- primIO $ prim__ptrWrite8 (prim__offsetPtr ptr (cast len)) 0
```

The leaked `wgpu` resources were caused by forgotten destructors. Detailed in Section 2.2.5, objects returned by `wgpu` functions need to call a destructor function once no longer needed. The destructor updates a reference count to the object, deallocating if no references are left. This construct does not exist in the C language (yet? [50]). Thus, only documentation can communicate these requirements to a programmer. The implemented fix for this issue is adding the destructor calls manually or with `ScopedIO`'s `defer` function. However, this introduces the potential for use-after-frees since the return values are not protected by `Refs`. Potential mechanisms for safe, ie. without potential use-after-free, and automatic destructors is discussed in Section 5.1.

No further errors were found via `valgrind`. However, `ScopedIO` was not perfect on its first try. Rather, fixing the issues that manifested as visible runtime errors (ie. segmentation faults and failing allocator `assert`s) earlier in the development cycle resulted in a correct implementation of `ScopedIO`. This does not obviate the need for dynamic analysis from tools like `valgrind`. Not only is external code liable to errors but, as shown by both examples in this section, the type system cannot capture all correctness requirements and mistakes will slip through. Dynamic analysis proves valuable in catching such mistakes.

Ultimately, `System.ScopedIO` accomplishes the goal of enforcing memory safety where possible (ie. with `newRef` and `safeFFIDrop`), without significant additional burden to the developer.

3.2. Evaluation

The concrete goal of an embeded domain specific language for GPU-accelerated mathematics did not surpass the research stage. Possible design are discussed in Section 4.3. Despite this failure, the work in this thesis is not far from being built upon. While not formally evaluated here, `System.ScopedIO` can be used with minimal, but non-zero, changes to the Idris compiler.

4. Related Work

4.1. Memory Safety

In recent years, government and industry organizations have called for the computing community to adopt memory safe languages and approaches [17, 46, 51]. Rust [48] has been mentioned repeatedly in this document for good reason, it demonstrates an optimal combination of memory safety and performance which was not thought possible. Rust is not the first to work in this domain, inspired by Cyclone [33] a memory safe “dialect” of C. Since then, a similar endeavor is being developed, Fil-C [14], is a garbage-collected version of C with the goal of maximal compatibility with existing C code.

4.2. Boilerplate

An unexpected focus of this dissertation was on boilerplate and exploring the various approaches that could be described as code or proof generation.

4.2.1. Damas-Hindely-Milnder Type Inference

While I was not aware of the connection before building `System.ScopedIO`, it turns out the connection between meta-programming and type inference was motivating factor in the creation of ML.⁷

“When Milner began his project to develop the Edinburgh LCF system, his plan was to replace Lisp, the metalanguage in the first generation Stanford LCF system, with a logically secure metalanguage, where a bug in the metalanguage code could not lead to an invalid proof.”

This work by Milner eventually led to the type system which underlies Haskell and Rust, dubbed Damas-Hindley-Milner type system. The core of this type system is a decideable inference algorithm which frees programmers from the need explicitly write type annotations. This feature is a ancestor of Idris’ `auto` implicits by way of *type class resolution* [18, 52].

4.2.2. Contemporary Approaches to Code Generation

Modern approaches tend to fall into three categories:

- *Macros* or *meta-programming* where the *abstract syntax tree* is manipulated and transformed.
- *Program synthesis* [19] which exhaustively searches for a program given examples input/output pairs or a specification.
- What I call “program generation” or the approach so simple it doesn’t have name: a script or program which outputs source code, usually from a configuration or data file.

`auto` implicit search falls most closely to program synthesis, though it should be noted, Idris has a seprate program synthesizer in its `:ps` REPL command. This implements type-driven program synthesis [54]. Another approach rooted in type theory is called *staging* [41].

The following sections provide more detail in the state-of-the art for macros and program synthesis.

⁷I probably should have realized this earlier, having known the name ML stood for “meta language” [45]

	Program Synthesis	Meta-programming	Binding Generation	Dep. Type Proof Search
Compiler Integration	Usually external tools	✓	External Tools	✓
Correctness/ Verification	✓	Generally none beyond host language, problems such as <i>macro hygiene</i> [40] exist, some approaches guarantee well typed macros [38]	Generally none beyond target language	✓
Generality	Synthesis tools tend to be domain specific, domain logic needs to be implemented	✓	Only for FFI	✓

Table 2: An overview of the design space for boilerplate-avoiding methods

4.2.2.1. Macros

Early Haskell proposals [21, 57] laid robust rules for FFI but did not commit to a marshalling approach. Modern approaches such as [20] still follow the principle as the original proposal of generating Haskell code using external tools.

An approach with similar ergonomics as the use of auto-implicits is `derive-storable` [39] module which uses the `DeriveGeneric` Haskell language extension⁸. `Generic` in Haskell is a metaprogramming approach which resembles macros.

Idris itself has a mechanism resembling macros, called “elaborator reflection” [22]. In elaborator reflection, user code can create and manipulate Idris’ internal representation of programs [32].

4.2.2.2. Program Synthesis

Program synthesis is an established area of research on program generation. Some of these approaches incorporate external tooling or languages for specifying the desired behavior of a program. Specification may be provided:

- “by-example” as actions to be recorded [24] or specific input-output pairs [28]
- as a specification, either specified in a domain-specific logic language [58] or in the host language [29].

The system presented in [29] integrates naturally with non-generated source code via Rust’s procedural macros; further, the synthesizer presented leverages Rust’s strong types for mini-

⁸It also provides an optional compiler plugin as a workaround for performance issues

mizing the program search space and implements a logic system incorporating Rust-specific safety rules.

Similar to [29], LiquidHaskell [61] allows programmers to annotate Haskell functions with properties to be verified through refinement types [30]. LiquidHaskell only verifies the written implementation matches a specification; though, the annotations can provide information for synthesis, it is not explored much [55].

4.3. GPU-Accelerated Mathematics

My original intention with the dissertation was to build an embedded domain-specific language (EDSL) for mathematics within Idris. The idea was inspired by various approaches to formalizing *automatic differentiation* [16, 42, 60, 62] within functional programming concepts.

This EDSL could then be compiled into a GPU shader program or executed in a GPU byte code interpreter, as in [37]. This is not particularly performant but would allow, for example, GPU acceleration in a dynamic environment like a REPL without the cost of shader recompilation.

5. Future work

5.1. Safe Destructors

As is, a `Ref` created from `safeFFIDrop` can be used in another call to `safeFFI` on the destructor again. Idris’ implements linear types or *multiplicities* which allow the programmer to specify a value should be used 0 or 1 times at runtime [15]. This can be used to ensure a destructor-requiring calls its destructor exactly once. However, only pushes off the primary issue that C does not have a way of distinguishing a destructor from a regular function. That is, with multiplicities, how does a destructor consume the single value while a regular function does not?

5.2. Performance

As described, functions like `sizeof` need to recalculate sizes each time they are called, however Idris’ multiplicities may be used again [15], this time to force these computations to happen 0 times at runtime, and only at compile time.

Another avenue for better performance is the use of lifetimes and borrowing. Rust differentiates `&` from `&mut` not only for correctness but also for performance [36]. In particular, the optimizer can better reason about *aliasing* [43], which enables compiler optimizations [31]. `ScopedIO` does not distinguish between mutable and immutable `Refs`, thus alias analysis-based optimizations cannot occur. A fuller version of the borrowing system has been implemented in Haskell [49] which can serve as inspiration.

Compiling this with a backend like `RefC` or the LLVM backend [34] would make Idris a high level interface to C. A programmer could gradually push down performance critical code into manual memory management, ie. not garbage collected, and explicit layouts within Idris before needing to fully break into C code.

6. Appendix

6.1. Code Listings

6.1.1. Struct marshaling

Implementation for `marshalStructBase` and `marshalStructRec`

```
(.ctype) : Marshal a' ity cty -> CType cty
(.ctype) (MkMarshal { repr } _) = repr

padto : AnyPtr -> CType a -> Int64
padto ptr repr =
  let ptrBits = the Int64 $ believe_me ptr
  in cast $ (doAlign (cast ptrBits) (cast $ alignof repr)) - (cast ptrBits)
  where
    -- https://github.com/libffi/libffi/blob/c93f9428d17cde4eb35517b58feeae6fb43aba5b/
    include/ffi_common.h#L118
    doAlign : Bits64 -> Bits64 -> Bits64
    doAlign v a = ((v-1) .|. (a-1))+1

marshalStructBase ma = MkMarshal $ \x, ptr =>
  let MkMarshal f = ma
  in f x (prim__castPtr $ prim__forgetPtr ptr)

marshalStructRec ma mb = MkMarshal $ \(a, bs), ptr => do
  let anyptr = prim__forgetPtr ptr
  let pad = padto anyptr ma.ctype
  let MkMarshal maf = ma
  let MkMarshal mbf = mb
  siz <- (+ pad) <$> maf a (prim__castPtr $ prim__offsetPtr anyptr pad)
  (+ siz) <$> mbf bs (prim__castPtr $ prim__offsetPtr anyptr $ cast siz)
```

6.2. Example.TriangleSys

A code snippet from the demo program used in Section 3.

```
render :
  (0 b' : Lifetime) ->
  (Ref a' GLFWwindow) ->
  (Ref a' (UnPtr WGPUAdapter)) ->
  (Ref a' (UnPtr WGPUSurface)) ->
  (Ref a' WGPUTexture) ->
  (Ref a' (UnPtr WGPUDevice)) ->
  (Ref a' (UnPtr WGPUQueue)) ->
  (Ref a' (UnPtr WGPURenderPipeline)) ->
  { auto 0 p : AtLeastAsLong a' b' } ->
  ScopedIO b' ()
render b' window adapter surface texture device queue pipeline = do
  enc <-
    safeFFIDrop
    wgpuDeviceCreateCommandEncoder
    (primIO . wgpuCommandEncoderRelease)
    [ shortenRef b' device
    , !(newRef
      { cty = WGPUCommandEncoderDescriptor }
      ( mkNULL WGPUChainedStruct
      , !(wgpuStringRef "command-encoder")
```

```

    ))
  ]

  putStrLn $ "wgpuTextureCreateView"

  view <- safeFFIDrop
    wgpuTextureCreateView
    (primIO . wgpuTextureViewRelease)
    [ !(getPtr (shortenRef b' texture))
      , mkNULL WGPUTextureViewDescriptor
    ]

  putStrLn $ "wgpuCommandEncoderBeginRenderPass"

  -- _ <- safeFFI prim__printf [ !(wgpuStringRef "\n\nrender-pass\n\n %d") ]

  pass <- safeFFIDrop
    wgpuCommandEncoderBeginRenderPass
    (primIO . wgpuRenderPassEncoderRelease)
    [ enc
      , !(newRef
        { cty = WGPURenderPassDescriptor }
        ( mkNULL WGPUChainedStruct
          , !(wgpuStringRef "render-pass")
          , 1
          , !(newRef
            { cty = WGPURenderPassColorAttachment }
            ( mkNULL WGPUChainedStruct
              , view
              , 0xffffffff -- depth slice undefined macro
              , the WGPUTextureView NULL
              , WGPULoadOp_Load
              , WGPUStoreOp_Store
              , !(newRef
                { cty = WGPUColor }
                (0.0, 1.1, 0.0, 1.0))
              ))
            , mkNULL WGPURenderPassDepthStencilAttachment
            , the WGPUQuerySet NULL
            , mkNULL WGPURenderPassTimestampWrites
          ))
        )
      ]

  putStrLn $ "wgpuRenderPassEncoderSetPipeline"

  safeFFI
    wgpuRenderPassEncoderSetPipeline
    [pass, shortenRef b' pipeline]

  putStrLn $ "wgpuRenderPassEncoderDraw"

  safeFFI
    wgpuRenderPassEncoderDraw
    [ pass
      , 3, 1
      , 0, 0
    ]

```



```

putStrLn $ "wgpuRenderPassEncoderEnd"

safeFFI wgpuRenderPassEncoderEnd [pass]

putStrLn $ "wgpuCommandEncoderFinish"

cbd <- newRef
  { cty = WGPUCommandBufferDescriptor }
  ( mkNULL WGPUChainedStruct
    , dbg' (dbg !wgpuStringNULL) -- !(wgpuStringRef "command-buffer")
    )

putStrLn $ "after cbd"
putStrLn $ "dbg enc"
_ <- pure $ dbg enc
putStrLn $ "after dbg enc"

buf <- safeFFIDrop
  wgpuCommandEncoderFinish
  (primIO . wgpuCommandBufferRelease)
  [ enc
    , cbd
  ]

putStrLn $ "new buffer"

bufbuf <- newRef { cty = WGPUCommandBuffer } buf

putStrLn "wgpuQueueSubmit"
safeFFI
  wgpuQueueSubmit
  [ shortenRef b' queue
    , 1
    , bufbuf
  ]

_ <- safeFFI wgpuSurfacePresent [shortenRef b' surface]

pure ()

```

References

- [1] Index - Erlang/OTP. Retrieved August 9, 2026 from <https://www.erlang.org/>
- [2] Pointer Declaration - Cppreference.Com. Retrieved August 3, 2026 from <https://en.cppreference.com/c/language/pointer>
- [3] MaybeUninit in Std::Mem - Rust. Retrieved August 3, 2026 from <https://doc.rust-lang.org/stable/std/mem/union.MaybeUninit.html>
- [4] Leaking - The Rustonomicon. Retrieved July 25, 2026 from <https://doc.rust-lang.org/nomicon/leaking.html>
- [5] Free(3): Allocate/Free Dynamic Memory - Linux Man Page. Retrieved August 1, 2026 from <https://linux.die.net/man/3/free>
- [6] Subtyping and Variance - The Rustonomicon. Retrieved July 12, 2026 from <https://doc.rust-lang.org/nomicon/subtyping.html>
- [7] Sealed Classes. Retrieved July 25, 2026 from <https://docs.oracle.com/en/java/javase/17/language/sealed-classes-and-interfaces.html>
- [8] A Haskell Foreign Function Interface. Retrieved July 25, 2026 from <https://www.haskell.org/definition/ffi/>
- [9] Drop in Std::Ops - Rust. Retrieved July 25, 2026 from <https://doc.rust-lang.org/stable/std/ops/trait.Drop.html>
- [10] Interfaces — Idris 2.0 Documentation. Retrieved August 8, 2026 from <https://idris2.readthedocs.io/en/latest/tutorial/interfaces.html#determining-parameters>
- [11] Valgrind Home. Retrieved August 8, 2026 from <https://valgrind.org/>
- [12] Valgrind. Retrieved August 8, 2026 from <https://valgrind.org/docs/manual/mc-manual.html>
- [13] Valgrind. Retrieved August 8, 2026 from <https://valgrind.org/docs/manual/manual-core.html#manual-core.mallocopts>
- [14] Fil-C. Retrieved August 1, 2026 from <https://fil-c.org/>
- [15] Multiplicities — Idris 2.0 Documentation. Retrieved July 25, 2026 from <https://idris2.readthedocs.io/en/latest/tutorial/multiplicities.html>
- [16] Atılım Gunes Baydin, Barak A Pearlmutter, Alexey Andreyevich Radul, and Jeffrey Mark Siskind. Automatic Differentiation in Machine Learning: A Survey.
- [17] Bob Lord. 2023. The Urgent Need for Memory Safety in Software Products. Retrieved July 21, 2026 from <https://www.cisa.gov/news-events/news/urgent-need-memory-safety-software-products>
- [18] Edwin Brady. 2013. Idris, a General-Purpose Dependently Typed Programming Language: Design and Implementation. *Journal of Functional Programming* 23, 5 (September 2013), 552–593. <https://doi.org/10.1017/S095679681300018X>
- [19] J Richard Buchi and Lawrence H Landweber. Solving Sequential Conditions by Finite-State Strategies.
- [20] Travis Cardwell and Sam Derbyshire. 2025. Automatic C Bindings Generation for Haskell. (2025).

- [21] Manuel Chakravarty, Sigbjorn Finne, Fergus Henderson, Marcin Kowalczyk, Daan Leijen, Simon Marlow, Erik Meijer, Sven Panne, BetaResearch GmbH, Simon Peyton Jones, Alastair Reid, Reid Consulting, Malcolm Wallace, and Michael Weber. The Haskell 98 Foreign Function Interface 1.0 An Addendum to the Haskell 98 Report.
- [22] David Christiansen and Edwin Brady. 2016. Elaborator Reflection: Extending Idris in Idris. In *Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming*, September 04, 2016. ACM, Nara Japan, 284–297. <https://doi.org/10.1145/2951913.2951932>
- [23] David G. Clarke, John M. Potter, and James Noble. 1998. Ownership Types for Flexible Alias Protection. In *Proceedings of the 13th ACM SIGPLAN Conference on Object-oriented Programming, Systems, Languages, and Applications*, October 1998. ACM, Vancouver British Columbia Canada, 48–64. <https://doi.org/10.1145/286936.286947>
- [24] Allen Cypher. 1991. EAGER: Programming Repetitive Tasks by Example. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems Reaching through Technology - CHI '91*, 1991. ACM Press, New Orleans, Louisiana, United States, 33–39. <https://doi.org/10.1145/108844.108850>
- [25] Ole-Johan Dahl, Bjørn Myhrhaug, and Kristen Nygaard. 1970. *Common Base Language*. Norwegian Computing Center Oslo.
- [26] Sergey "Shnatsel" Davidoff. 2025. How to Avoid Bounds Checks in Rust (without Unsafe!). Retrieved August 2, 2026 from <https://shnatsel.medium.com/how-to-avoid-bounds-checks-in-rust-without-unsafe-f65e618b4c1e>
- [27] Edwin Brady. 2026. Dissertation Meeting - Auto Implicits and Exhaustiveness.
- [28] Kasra Ferdowsifard, Allen Ordoorkhanians, Hila Peleg, Sorin Lerner, and Nadia Polikarpova. 2020. Small-Step Live Programming by Example. In *Proceedings of the 33rd Annual ACM Symposium on User Interface Software and Technology*, October 20, 2020. ACM, Virtual Event USA, 614–626. <https://doi.org/10.1145/3379337.3415869>
- [29] Jonáš Fiala, Shachar Itzhaky, Peter Müller, Nadia Polikarpova, and Ilya Sergey. 2023. Leveraging Rust Types for Program Synthesis. *Proceedings of the ACM on Programming Languages* 7, PLDI (June 2023), 1414–1437. <https://doi.org/10.1145/3591278>
- [30] Tim Freeman and Frank Pfenning. 1991. Refinement Types for ML. *ACM SIGPLAN Notices* 26, 6 (June 1991), 268–277. <https://doi.org/10.1145/113446.113468>
- [31] Rakesh Ghiya, Daniel Lavery, and David Sehr. On the Importance of Points-To Analysis and Other Memory Disambiguation Methods For C Programs.
- [32] Stefan Höck. 2026. Stefan-Hoeck/Idris2-Elab-Util. Retrieved August 11, 2026 from <https://github.com/stefan-hoeck/idris2-elab-util>
- [33] Trevor Jim, Greg Morrisett, Dan Grossman, Michael Hicks, James Cheney, and Yanling Wang. Cyclone: A Safe Dialect of C.
- [34] Johann Rudloff. Rapid - Native Compiler for Idris 2. Retrieved August 1, 2026 from <https://rapid.sinyax.net/>
- [35] Simon L Peyton Jones and John Launchbury. 1991. Unboxed Values as First Class Citizens in a Non-Strict Functional Language. In *Conference on Functional Programming Languages and Computer Architecture*, 1991. Springer, 636–666.

- [36] Ralf Jung, Hoang-Hai Dang, Jeehoon Kang, and Derek Dreyer. 2020. Stacked Borrows: An Aliasing Model for Rust. *Proceedings of the ACM on Programming Languages* 4, POPL (January 2020), 1–32. <https://doi.org/10.1145/3371109>
- [37] Matthew J. Keeter. 2020. Massively Parallel Rendering of Complex Closed-Form Implicit Surfaces. *ACM Transactions on Graphics* 39, 4 (August 2020). <https://doi.org/10.1145/3386569.3392429>
- [38] Oleg Kiselyov. 2014. The Design and Implementation of BER MetaOCaml: System Description. In *Functional and Logic Programming*, Michael Codish, Eijiro Sumii, David Hutchison, Takeo Kanade, Josef Kittler, Jon M. Kleinberg, Alfred Kobsa, Friedemann Mattern, John C. Mitchell, Moni Naor, Oscar Nierstrasz, C. Pandu Rangan, Bernhard Steffen, Demetri Terzopoulos, Doug Tygar and Gerhard Weikum (eds.). Springer International Publishing, Cham, 86–102. https://doi.org/10.1007/978-3-319-07151-0_6
- [39] Mateusz Kłoczko. 2024. Mkloczko/Derive-Storable. Retrieved July 25, 2026 from <https://github.com/mkloczko/derive-storable>
- [40] Eugene Kohlbecker, Daniel P. Friedman, Matthias Felleisen, and Bruce Duba. 1986. Hygienic Macro Expansion. In *Proceedings of the 1986 ACM Conference on LISP and Functional Programming - LFP '86*, 1986. ACM Press, Cambridge, Massachusetts, United States, 151–161. <https://doi.org/10.1145/319838.319859>
- [41] András Kovács. 2022. Staged Compilation with Two-Level Type Theory. *Proceedings of the ACM on Programming Languages* 6, ICFP (August 2022), 540–569. <https://doi.org/10.1145/3547641>
- [42] Faustyna Krawiec, Simon Peyton Jones, Neel Krishnaswami, Tom Ellis, Richard A. Eisenberg, and Andrew Fitzgibbon. 2022. Provably Correct, Asymptotically Efficient, Higher-Order Reverse-Mode Automatic Differentiation. *Proceedings of the ACM on Programming Languages* 6, POPL (January 2022), 1–30. <https://doi.org/10.1145/3498710>
- [43] William Landi and Barbara G. Ryder. 1991. Pointer-Induced Aliasing: A Problem Taxonomy. In *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages - POPL '91*, 1991. ACM Press, Orlando, Florida, United States, 93–103. <https://doi.org/10.1145/99583.99599>
- [44] Barbara Liskov. 1996. A History of CLU. In *History of Programming Languages—II*, Thomas J. Bergin and Richard G. Gibson (eds.). ACM, New York, NY, USA, 471–510. <https://doi.org/10.1145/234286.1057826>
- [45] David MacQueen, Robert Harper, and John Reppy. 2020. The History of Standard ML. *Proceedings of the ACM on Programming Languages* 4, HOPL (June 2020), 1–100. <https://doi.org/10.1145/3386336>
- [46] Maddie Stone. 2022. The More You Know, The More You Know You Don’t Know. Retrieved July 21, 2026 from <https://projectzero.google/2022/04/the-more-you-know-more-you-know-you.html>
- [47] Marina Abramović and Ulay. 1980. Rest Energy.
- [48] Nicholas D. Matsakis and Felix S. Klock. 2014. The Rust Language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, October 18, 2014. ACM, Portland Oregon USA, 103–104. <https://doi.org/10.1145/2663171.2663188>

- [49] Yusuke Matsushita and Hiromi Ishii. 2026. Pure Borrow: Linear Haskell Meets Rust-Style Borrowing. *Proceedings of the ACM on Programming Languages* 10, PLDI (June 2026), 303–327. <https://doi.org/10.1145/3808259>
- [50] JeanHeyd Meneide. 2025. Programming Languages — C — Defer, a Mechanism for General Purpose, Lexical Scope-Based Undo.
- [51] Microsoft Security Response Center Team. 2019. A Proactive Approach to More Secure Code. Retrieved July 21, 2026 from <https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>
- [52] John Peterson and Mark Jones. 1993. Implementing Type Classes. *ACM SIGPLAN Notices* 28, 6 (June 1993), 227–236. <https://doi.org/10.1145/173262.155112>
- [53] L Peyton. Lazy Functional State Threads.
- [54] Nadia Polikarpova, Ivan Kuraj, and Armando Solar-Lezama. 2016. Program Synthesis from Polymorphic Refinement Types. In *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation*, June 02, 2016. ACM, Santa Barbara CA USA, 522–538. <https://doi.org/10.1145/2908080.2908093>
- [55] Patrick Redmond, Gan Shen, and Lindsey Kuper. 2021. Toward Hole-Driven Development with Liquid Haskell. <https://doi.org/10.48550/arXiv.2110.04461>
- [56] Ron Morrison, Fred Brown, Richard Connor, Quintin Cutts, Alan Dearle, Graham Kirby, and Dave Munro. Napier88 Reference Manual Release 2.2.1.
- [57] Simon Peyton Jones. Tackling the Awkward Squad: Monadic Input/Output, Concurrency, Exceptions, and Foreign-Language Calls in Haskell.
- [58] Armando Solar-Lezama. 2013. Program Sketching. *International Journal on Software Tools for Technology Transfer* 15, 5–6 (October 2013), 475–495. <https://doi.org/10.1007/s10009-012-0249-7>
- [59] L. S. Tang. 1991. C++'s Destructors Can Be Destructive. *ACM SIGPLAN Notices* 26, 10 (October 1991), 44–52. <https://doi.org/10.1145/122616.122622>
- [60] Matthijs Vákár and Tom Smeding. 2022. CHAD: Combinatory Homomorphic Automatic Differentiation. *ACM Transactions on Programming Languages and Systems* 44, 3 (September 2022), 1–49. <https://doi.org/10.1145/3527634>
- [61] Niki Vazou, Eric L Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton-Jones. Refinement Types For Haskell.
- [62] R. E. Wengert. 1964. A Simple Automatic Derivative Evaluation Program. *Communications of the ACM* 7, 8 (August 1964), 463–464. <https://doi.org/10.1145/355586.364791>